

Sistema de Hospedagem

Documento Técnico — Sprint 4

Evolução Arquitetural e Padrões de Projeto

Disciplina:	Programação Modular
Curso:	Engenharia de Software — PUC Minas
Período:	1º Semestre / 2026

1. Funcionalidades Escolhidas

Para a Sprint 4 foram selecionadas as opções **3 – Central de Notificações** e **4 – Integração com Múltiplos Meios de Pagamento**, além da aplicação obrigatória do padrão **Singleton** em um componente de infraestrutura transversal ao sistema.

1.1 Central de Notificações (Opção 3)

O sistema passa a notificar clientes e proprietários automaticamente quando eventos relevantes ocorrem: criação de reserva, cancelamento, check-in, check-out e confirmação de pagamento. As notificações são enviadas por múltiplos canais (e-mail, SMS e notificação interna), podendo novos canais ser adicionados sem qualquer alteração no código existente.

1.2 Múltiplos Meios de Pagamento (Opção 4)

O sistema passa a suportar PIX, cartão de crédito, cartão de débito, dinheiro e carteiras digitais (PicPay, Mercado Pago, PagBank). Cada meio possui suas próprias regras de validação e processamento, e novos meios podem ser incorporados sem modificar o fluxo de pagamento existente.

2. Solução Proposta: Padrões de Projeto

2.1 Observer — Central de Notificações

O padrão Observer define uma dependência um-para-muitos: quando o *Subject* muda de estado, todos os seus *Observers* são notificados automaticamente. No sistema, a **CentralNotificacoes** atua como Subject e os canais de comunicação atuam como Observers.

Estrutura implementada:

Elemento	Papel no Observer
CanalNotificacao	Interface Observer — define o contrato notificar()
CanalEmail	Observer concreto — envia e-mail ao cliente
CanalSms	Observer concreto — envia SMS ao cliente
CanalNotificacaoInterna	Observer concreto — armazena notificação no sistema
CentralNotificacoes	Subject — gerencia canais e publica eventos
ContextoNotificacao	Objeto de dados rico carregado junto ao evento
EventoHospedagem	Enum com os tipos de eventos suportados

Trecho representativo:

```
centralNotificacoes.publicar(new ContextoNotificacao(  
    EventoHospedagem.RESERVA_CRIADA, salvo.getId(),  
    salvo.getCliente().getNome(), email, telefone, detalhes));
```

2.2 Strategy — Múltiplos Meios de Pagamento

O padrão Strategy define uma família de algoritmos, encapsula cada um e os torna intercambiáveis. O contexto (**ProcessadorPagamento**) delega o processamento à estratégia selecionada em tempo de execução, sem conhecer os detalhes de cada meio.

Estrutura implementada:

Elemento	Papel no Strategy
MeioPagamento	Interface Strategy — getNome(), validar(), processar()
PagamentoPix	Estratégia concreta — valida chave PIX
PagamentoCartaoCredito	Estratégia concreta — valida 16 dígitos, calcula juros
PagamentoCartaoDebito	Estratégia concreta — aplica desconto de 2%
PagamentoDinheiro	Estratégia concreta — sempre válida, sem acréscimos
PagamentoCarteiraDigital	Estratégia concreta — valida carteira suportada
ProcessadorPagamento	Contexto — mantém mapa de estratégias e delega execução
DadosPagamento	VO com dados específicos de cada meio
ResultadoPagamento	VO com código de transação, valor e status

Trecho representativo:

```
ResultadoPagamento resultado = processadorPagamento  
.processar(nomeMeio, aluguel.getValorFinal(), dados);
```

3. Justificativa das Escolhas

Observer — Central de Notificações

O sistema possuía um único ponto de saída para comunicação com o cliente (o formulário de aluguel em texto puro). A expansão para múltiplos canais criaria acoplamento direto entre o serviço de aluguel e cada canal. O Observer elimina esse acoplamento: **AluguelService** publica um evento e desconhece completamente quem o recebe. Adicionar o canal WhatsApp no futuro, por exemplo, requer apenas implementar **CanalNotificacao** e registrá-lo na central.

Strategy — Meios de Pagamento

O sistema anterior possuía um único objeto **Pagamento** sem lógica de processamento diferenciada. Adicionar meios de pagamento com if-else ou switch na camada de serviço violaria o princípio Aberto/Fechado. O Strategy encapsula cada meio como uma classe independente com validação e processamento próprios. O **ProcessadorPagamento** funciona como um registro extensível: novos meios se auto-registram sem modificar o contexto.

4. Utilização do Singleton — GerenciadorLogs

Componente: GerenciadorLogs

O **GerenciadorLogs** implementa o padrão Singleton com double-checked locking para garantir segurança em ambientes multithreaded.

Justificativa da instância única:

- Logs são um recurso compartilhado globalmente — múltiplas instâncias gerariam registros fragmentados e inconsistentes.
- A lista de registros deve ser única e centralizada para garantir ordem cronológica e visibilidade completa.
- Componentes de camadas distintas (services, observer, strategy) precisam escrever no mesmo log sem acoplamento entre si.
- A criação de múltiplas instâncias desperdiçaria memória para algo que é intrinsecamente global.

Implementação (double-checked locking):

```
private static volatile GerenciadorLogs instancia;

public static GerenciadorLogs getInstance() {
    if (instancia == null) {
        synchronized (GerenciadorLogs.class) {
            if (instancia == null) {
                instancia = new GerenciadorLogs();
            }
        }
    }
    return instancia;
}
```

Integração com os demais padrões:

O GerenciadorLogs é utilizado pela **CentralNotificacoes** (registra publicação de eventos), pelo **ProcessadorPagamento** (registra cada transação com código e valor) e pelo **AluguelService** (registra criação e cancelamento de aluguéis), conectando os três padrões da Sprint 4 em uma infraestrutura de observabilidade coesa.

5. Correções Arquiteturais Realizadas

Duplicidade de serviço (ResidenciaService / ResidencialService)

Existiam duas classes quase idênticas — ResidenciaService (com tratamento de exceções correto) e ResidencialService (com RuntimeException genérico). ResidencialService foi removido; apenas ResidenciaService permanece.

Switch de tipo em QuartoService

O método resolverTipo() centralizava um switch para mapear String -> Class. Criou-se o enum TipoQuarto com os atributos discriminador e classe, eliminando o switch e delegando a resolução ao próprio domínio via TipoQuarto.fromString().

Taxa de berço como constante estática

QuartoDuplo.TAXA_BERCO era uma constante estática (public static final double = 25.0). Foi migrada para um atributo de instância com coluna JPA (@Column name='taxa_berco'), permitindo que cada quarto duplo tenha sua própria taxa configurável.

Modularização do frontend

Sidebar e topbar eram copiados manualmente em cada página HTML. Foram extraídos para componentes JavaScript (sidebar.js, topbar.js) com funções renderSidebar() e renderTopbar(), injetados via module import. O módulo api.js centraliza todos os fetch calls. O toast.js oferece feedback visual reutilizável.

6. Benefícios da Nova Arquitetura

Benefício	Descrição
Extensibilidade	Novos canais de notificação e meios de pagamento são adicionados implementando uma interface.
Baixo acoplamento	AluguelService não conhece os canais nem os meios: depende apenas de CentralNotificacoes e Pr...
Responsabilidade única	Cada canal de notificação e cada meio de pagamento têm responsabilidade bem definida e isolada
Observabilidade	GerenciadorLogs (Singleton) captura todas as operações críticas de forma centralizada e cronológic...
Domínio mais rico	TipoQuarto encapsula a resolução de tipo. A taxa do berço virou atributo de domínio. O enum TipoQ...
Frontend desacoplado	Componentes JS reutilizáveis eliminam duplicação no HTML e centralizam acesso à API, facilitand...
Testabilidade	Observer e Strategy são facilmente testados em isolamento com stubs/mocks. O Singleton pode se